KU LEUVEN

Lab seminar

Sources, wiki and code in one folder

Doing research with AI agents



October 2026

Damien Bonnet-Eymard

FLANDERS MAKE
DRIVING INNOVATION IN MANUFACTURING

RELIABLE & ROBUST DESIGN

What research needs from an agent

1

Literature review

- Find and triage papers
- Build the reading list
- Follow who cites what

2

Paper digestion

- Methods, equations, values
- What we take, what we leave
- Compare across papers

3

Coding

- Implement and test methods
- Revive legacy code
- Reproduce published results

4

Reporting

- Papers and reports
- Slides (like these)
- Notebooks and figures

What they share: the agent must know your project (papers, conventions, past decisions). A chat window starts from zero every time.

Anatomy of an AI coding agent

Your framework *the memory*

Plain Markdown in your project folder: instructions, skills, sources, wiki.

AGENTS.md · .agents/skills/ · docs/wiki/ · resources/

Harness *the hands*

Reads and writes files, runs code, searches, loops until done. Where you talk to the agent.

Claude Code · Codex · Gemini CLI · OpenCode · Qwen Code

LLM *the brain*

Predicts the next token. Knows the world up to its cutoff, nothing about your project.

Claude · GPT · Gemini · Qwen · DeepSeek · Kimi

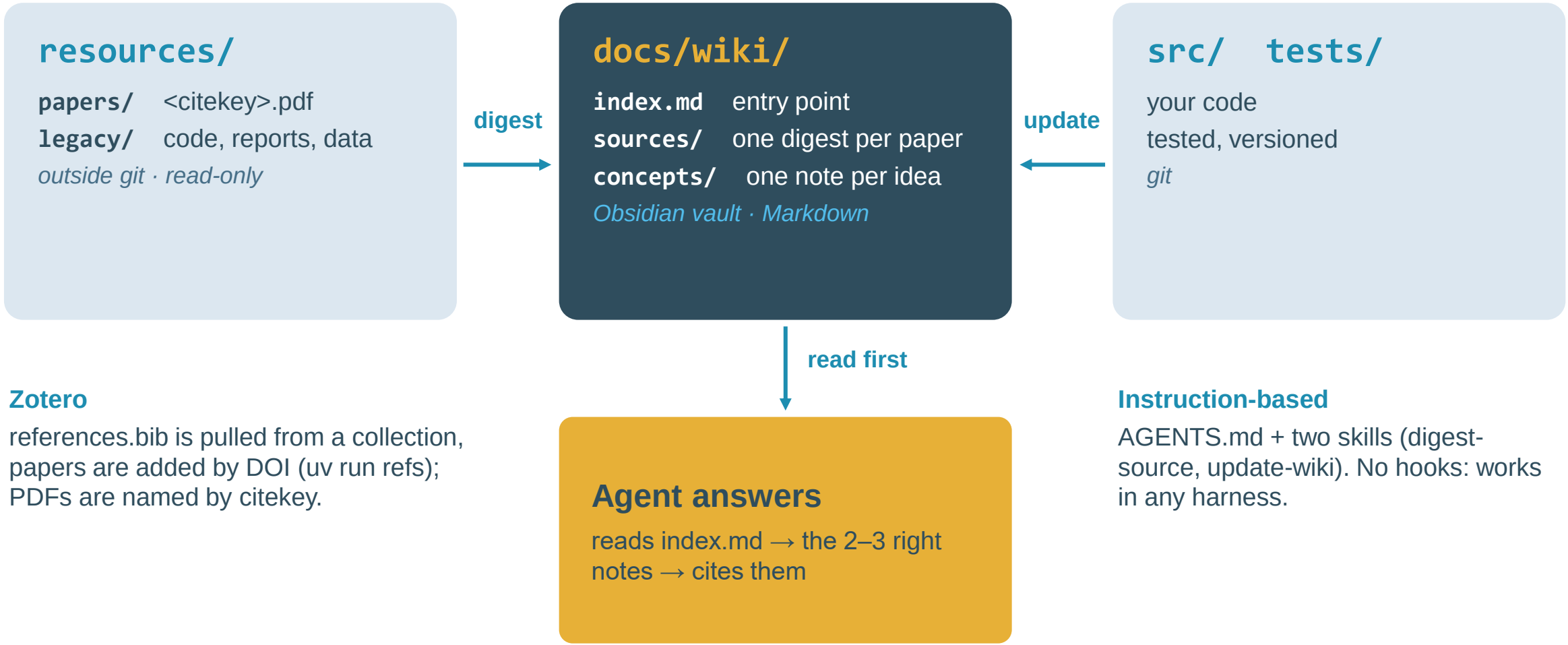
Own the top layer

Models and harnesses change every few months; so do the leaders.

Keep what you build in plain Markdown in your folder → **switch model or harness, keep your knowledge.**

AGENTS.md is read by most harnesses. Claude Code reads CLAUDE.md, which just imports it.

One folder: sources, wiki, code



Zotero

references.bib is pulled from a collection, papers are added by DOI (uv run refs); PDFs are named by citekey.

Instruction-based

AGENTS.md + two skills (digest-source, update-wiki). No hooks: works in any harness.

The wiki has two readers

You, in Obsidian

- Browse from index.md, follow [[links]]
- Graph view: how papers and concepts connect
- A literature memory that stays current
- “What did we take from X?” in seconds

The agent, as context

- AGENTS.md: read index.md before answering
- Lands on the 2–3 relevant notes, not 40 PDFs
- Answers cite the notes, so you can check
- Same memory in every session and harness

```
# Tian et al. (2025): hoop ply drop location
[@tianInfluenceCircumferentialPly2025]. Type IV, 30 L…
## What we take
- **Arc-type hoop ply drops.** …  → [[hoop-ply-drops]]
## What we don't take
Hashin progressive damage (excluded in v1).
```

One note per source or concept

A digest says what the project uses or rejects from the paper, with a link to where. A selection, not a summary.

Setting it up: Markdown that tells the agent how to work

```
AGENTS.md              always read
CLAUDE.md              → @AGENTS.md
.agents/skills/
  digest-source/       paper → wiki
  update-wiki/         code → wiki
docs/CONTEXT.md        vocabulary
docs/adr/              decisions
```

Skill = a Markdown recipe the agent loads only when the task matches its description.

AGENTS.md = always-on rules: layout, conventions, where things are.

You don't write them

"Set up this repo so every new paper gets digested into the wiki."

→ the agent writes AGENTS.md and the skill.

The hard part: alignment

- The agent does what you said, not what you meant.
- Early on you often don't know what you want: scope, conventions, what counts as done.

→ **make it ask you first**

Matt Pocock's skills: align before you build

`grill`
interview until you share the plan

→

`spec`
write the plan down

→

`tickets`
split it into small issues

→

`implement`
one issue at a time

`setup`

Run once per repo: where issues and domain docs (CONTEXT.md, ADRs) live. Start here.

`router`

Not sure which skill fits? Ask it: a router over all the skills.

In practice

A project: one spec per milestone, split into issues in .scratch/

This talk: planned in one grilling session, ~20 questions in 4 rounds

github.com/mattpocock/skills open source, plain Markdown, any harness. Names change between releases; the loop stays

A real project, a few weeks in

```
my-project/
├─ CLAUDE.md          rules, where things are
├─ references.bib
├─ src/  tests/
├─ .scratch/          m0 … m6: spec + issues
└─ docs/              Obsidian vault
   ├─ CONTEXT.md  roadmap.md  adr/
   └─ wiki/
      ├─ index.md
      ├─ sources/     10 digests
      ├─ concepts/    10 notes
      └─ reading-list.md

resources/  outside git: papers/, legacy/
```

LIVE DEMO

Burst reliability of composite pressure vessels

1. CLAUDE.md: the rule that builds the wiki
2. index.md → “modelling-chain: start here”
3. A digest: what we take / don't take
4. Graph view in Obsidian
5. Ask a question → the answer cites notes

From scratch in 15 minutes

Cantilever tip deflection $\delta = PL^3 / 3EI$ with E, I, P uncertain: Monte Carlo vs polynomial chaos, Sobol indices. A closed-form reference to check the agent against.

LIVE DEMO

1. `Copy the template, name the Zotero collection in resources.yaml`
2. `Interview me: compare Monte Carlo and PCE for a cantilever's tip deflection, with E, I and P uncertain…`
3. `Find ~5 key papers on PCE and Sobol sensitivity, add them with uv run refs add.` → `uv run fetch` → “Digest the new papers.”
4. `Implement MC and PCE (chaospy), check mean and variance against the analytical result, compute Sobol indices, then update the wiki.`
5. Open docs/ in Obsidian: index, a digest, the graph.

Limits and risks

Invented citations and numbers

LLMs produce plausible references and values that don't exist. Check against the PDF; a digest points to the passage, so it's cheap.

Confidentiality

Prompts and files go to the model provider. Unpublished data, reviews, partner documents: check your institution's and project rules first.

Code that runs isn't code that's right

Ask for tests and checks against known solutions, and read the diff. You remain the author.

Cost and lock-in

Subscriptions and API usage cost money, and quotas change. Plain Markdown keeps your work portable across providers.

Navier–Stokes: the headline without the story

8 Sep 2026 OpenAI announces an AI-generated proof that the 3D Navier–Stokes equations with a smooth force can blow up in finite time: ~10,000 agents in parallel, formalised in Lean. Mathematicians are still digesting it.

"The actual solving of these problems is only a proxy goal for the primary goal of developing mathematical understanding."

Terence Tao, on the human Alpöge–Buckmaster result, 7 Sep 2026

A "severe misalignment" between AI companies and mathematics: the "mass production" of proofs endangers "conceptual understanding and insight".

Declaration signed by 25 Fields medallists, 12 Sep 2026

"A proof that is correct but incomprehensible … gives us the headline without the story."

Tapio Schneider, guest post on Tao's blog, 23 Sep 2026

NPR, 22 Sep: *"AI solved one of math's hardest problems. Humanity learned nothing (so far)."*

Sources: T. Tao, terrytao.wordpress.com (7 and 23 Sep 2026); Scientific American, "25 winners of math's 'Nobel Prize' decry the AI invasion of their discipline"; NPR (22 Sep 2026).

The value is in your head

A result nobody understands, remembers or communicates is wasted computation.

The acceleration is real: weeks of work can be redone in an afternoon. That can make past results feel obsolete, and it's overwhelming.

What keeps its value: your understanding, your judgement, the questions you ask, and the people you tell.

Use the agent to learn

- Ask it to explain, then explain it back
- Derive by hand, then let it check you
- Read the digest, then the paper
- `/teach`: a skill that teaches you a concept inside your project
- Let a grilling skill question your plan and your reasoning
- The wiki is written for you too: read it

Start tomorrow

1 Clone the template

github.com
bonneted/
research-agent-template

2 Install the tools

Claude Code or another agent, uv, Obsidian; Matt Pocock's skills (optional)

3 Add papers, paste the prompt

A Zotero collection, then uv run fetch

```
This is a new research project. Interview me, one question at a time, about the
research question, the scope and what I already know, until you can write the project
line in AGENTS.md and the first terms in docs/CONTEXT.md. Then digest every resource
(digest-source skill), and propose a first plan as small, testable steps.
```

Start small: one project, five papers, one question.

Damien Bonnet-Eymard
KU Leuven



The project leading to this application has received funding from the European Union's Horizon 2020 research and innovation program under the Marie Skłodowska-Curie grant agreement No 955393.

FLANDERS MAKE
DRIVING INNOVATION IN MANUFACTURING